

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT
on

OPERATING SYSTEMS

Submitted by

NYDHILE NAGARAJ (1WA24CS195)

in partial fulfilment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “OPERATING SYSTEMS – 23CS4PCOPS” carried out by NYDHILE NAGARAJ (1WA24CS195), who is Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026. The Lab report has been approved as it satisfies the academic requirements in respect of an OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Dr. Seema Patil
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl No	Experiment Title	Page No
1	Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. (Any one) a) FCFS b) SJF c) Priority d) Round Robin (Experiment with different quantum sizes for RR algorithm)	1-35
2	Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.	36-44
3	Write a C program to simulate Real-Time CPU Scheduling algorithms: (Any one) a) Rate-Monotonic b) Earliest-Deadline First c) Proportional Scheduling	45-61
4	Write a C program to simulate: (Any one) a) Producer-Consumer problem using semaphores b) Dining-Philosopher's problem	62-75
5	Write a C program to simulate: (Any one) a) Bankers' algorithm for the purpose of deadlock avoidance b) Deadlock Detection	76-94
6	Write a C program to simulate the following contiguous memory allocation techniques. (Any one) a) Worst-Fit b) Best-Fit c) First-Fit	95-105
7	Write a C program to simulate page replacement algorithms. (Any one) a) FIFO b) LRU c) Optimal	106-119

Course Outcomes

C01	Apply the different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating System.
C04	Conduct practical experiments to implement the functionalities of Operating system.

Program -1

Question:

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. (Any one)

- a) FCFS
- b) SJF
- c) Priority
- d) Round Robin (Experiment with different quantum sizes for RR algorithm)

FCFS

CODE:

```
#include <stdio.h>

int main()
{
printf("Name: NYDHILE NAGARAJ\nUSN:1WA24CS195\n\n");
    int n, i;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], ct[n], tat[n], wt[n];

    printf("Enter Arrival Time:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &at[i]);

    printf("Enter Burst Time:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &bt[i]);

    int t = 0;
```



```

for(i = 0; i < n; i++)
{
    if(t < at[i])
        t = at[i];
    ct[i] = t + bt[i];
    t = ct[i];
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i]; }
printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
for(i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, at[i], bt[i], ct[i], tat[i], wt[i]); }
return 0;}

```

Result:

```

C:\Users\nydhi\OneDrive\Doc
Name: NYDHILE NAGARAJ
USN: 1WA24CS195

Enter number of processes: 4
Enter Arrival Time:
0 0 0 0
Enter Burst Time:
7 3 4 6

Process AT      BT      CT      TAT      WT
P1      0      7      7      7      0
P2      0      3     10     10      7
P3      0      4     14     14     10
P4      0      6     20     20     14

Process returned 0 (0x0)   execution time : 19.596 s
Press any key to continue.
|

```

6/3/26

Date _____
Page 4

Lab Program 1

i) FCFS (with arrival time)

```
#include <stdio.h>
struct Process {
    int pid;
    int arrival;
    int burst;
    int completion;
    int waiting;
    int turnaround;
};

int main () {
    int n, i, j;
    printf ("Enter number of processes : ");
    scanf ("%d", &n);
    struct Process p[n], temp;

    for (i=0; i<n; i++) {
        printf ("Enter arrival & burst time\nfor process %d : ", i+1);
        p[i].pid = i+1;
        scanf ("%d %d", &p[i].arrival, &p[i].burst);
    }

    for (i=0; i<n-1; i++) {
        for (j=i+1; j<n; j++) {
            if (p[i].arrival > p[j].arrival) {
                temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }
}
```

```

int current_time = 0; float avg_tat = 0,
for (i = 0; i < n; i++) {      avg_wt = 0;
    if (current_time < p[i].arrival) {
        current_time = p[i].arrival;
    }

    p[i].completion = current_time + p[i].burst;
    p[i].turnaround = p[i].completion - p[i].arrival;
    p[i].waiting = p[i].turnaround - p[i].burst;
    avg_tat += p[i].turnaround; avgwt += p[i].waiting;
    current_time = p[i].completion;
}

```

```

printf("\n PID \t AT \t BT \t CT \t TAT \t
      WT \n");

```

```

for (i = 0; i < n; i++) {
    printf("%d \t %d \t %d \t %d \t %d \t
          %d \n",

```

```

        p[i].pid,
        p[i].arrival,
        p[i].burst,
        p[i].completion,
        p[i].turnaround,
        p[i].waiting);
}

```

```

printf("\n average TAT =
      %.2f", avg_tat / n);
printf("\n average WT = %.2f",
      avgwt / n);

```

OUTPUT:

Enter total number of processes : 4
process [1]

Arrival time : 0

Burst time : 2

Process [2]

Arrival time : 1

Burst time : 2

Process [3]

Arrival time : 5

Burst time : 3

Process [4]

Arrival time : 6

Burst time : 4

Process	Arrival time	Burst time	Completion time	Turnaround	Wait
1	0	2	2	2	0
2	1	2	4	3	1
3	5	3	8	3	0
4	6	4	12	6	2

Average Turnaround time : 3.50

Average Waiting time : 0.75

Ans
1/22/6

SJF (Non – Preemptive)

CODE:

```
#include <stdio.h>

int main()
{
printf("Name: NYDHILE NAGARAJ\nUSN:1WA24CS195\n\n");

    int n, i;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], ct[n], tat[n], wt[n];
    int done[n];

    printf("Enter Arrival Time:\n");
    for(i = 0; i < n; i++)
    {
        scanf("%d", &at[i]);
        done[i] = 0;
    }

    printf("Enter Burst Time:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &bt[i]);

    int t = 0, completed = 0;

    while(completed < n)
    {
        int min = 9999;
```

```

int index = -1;

for(i = 0; i < n; i++)
{
    if(at[i] <= t && done[i] == 0)
    {
        if(bt[i] < min)
        {
            min = bt[i];
            index = i;
        }
    }
}

if(index == -1)
{
    t++;
}
else
{
    ct[index] = t + bt[index];
    t = ct[index];

    tat[index] = ct[index] - at[index];
    wt[index] = tat[index] - bt[index];

    done[index] = 1;
    completed++;
}

printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");

```

```

for(i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
}

return 0;
}
Result:

```

```

C:\Users\nydhi\OneDrive\Doc
Name: NYDHILE NAGARAJ
USN:1WA24CS195

Enter number of processes: 4
Enter Arrival Time:
0 8 3 5
Enter Burst Time:
7 4 3 6

Process AT      BT      CT      TAT     WT
P1      0      7      7      7      0
P2      8      4      14     6      2
P3      3      3      10     7      4
P4      5      6      20     15     9

Process returned 0 (0x0)   execution time : 16.153 s
Press any key to continue.

```


ii) SJF Non Preemptive

```
#include <stdio.h>
int main () {
    int n, at[10], bt[10], ct[10], tat[10],
        wt[10], done[10];
    int i, time = 0, completed = 0, min, index;
    float avg_tat = 0, avg_wt = 0;

    printf("Enter no. of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        printf("Enter AT & BT for P%d: ", i+1);
        scanf("%d %d", &at[i], &bt[i]);
        done[i] = 0;
    }

    while (completed < n) {
        min = 999;
        index = -1;

        for (i = 0; i < n; i++) {
            if (at[i] <= time && done[i] == 0 &&
                bt[i] < min) {
                min = bt[i];
                index = i;
            }
        }
    }
```



```
if (index == -1) {
    time ++; }
else {
    ct[index] = time + bt[index];
    tat[index] = ct[index] - at[index];
    wt[index] = tat[index] - bt[index];
```

```
time = ct[index];
done[index] = 1;
completd ++;
```

```
avg-tat += tat[index];
avg-wt += wt[index];
}
}
```

```
printf("\n P \t AT \t BT \t CT \t TAT \t WT \n");
for (i=0; i<n; i++) {
    printf("P %d \t AT %d \t BT %d \t CT %d \t TAT %d \t WT %d \n", i+1, at[i], bt[i], ct[i], tat[i], wt[i]); }
```

```
printf("\n average TAT = %.2f", avg-tat/n);
printf("\n average WT = %.2f", avg-wt/n);
```

```
return 0;
}
```

iii) SJF preemptive

OUTPUT:

Enter number of processes:

2

Enter AT and BT for P1:

3 4

Enter AT and BT for P2:

5 6

P	AT	BT	CT	TAT	WT
P1	3	4	7	4	0
P2	5	6	13	8	2

Average TAT = 6.00

Average WT = 1.00

QW
6-5-22

iii) SJF preemptive

#include <stdio.h>

int main() {

int n, at[10], bt[10], rt[10], ct[10],
tat[10], wt[10];

int i, time = 0, completed = 0, min, index;

float avg-tat = 0, avg-wt = 0;

printf("Enter number of processes:");

scanf("%d", &n);

SRTF

CODE:

```
#include <stdio.h>

int main()
{
printf("Name: NYDHILE NAGARAJ\nUSN:1WA24CS195\n\n");
    int n, i;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], rt[n], ct[n], tat[n], wt[n];

    printf("Enter Arrival Time:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &at[i]);

    printf("Enter Burst Time:\n");
    for(i = 0; i < n; i++)
    {
        scanf("%d", &bt[i]);
        rt[i] = bt[i];
    }

    int t = 0, completed = 0;

    while(completed < n)
    {
        int min = 9999;
        int index = -1;
```

```

for(i = 0; i < n; i++)
{
    if(at[i] <= t && rt[i] > 0)
    {
        if(rt[i] < min)
        {
            min = rt[i];
            index = i;
        }
    }
}

if(index == -1)
{
    t++;
}
else
{
    rt[index]--;
    t++;

    if(rt[index] == 0)
    {
        ct[index] = t;
        completed++;

        tat[index] = ct[index] - at[index];
        wt[index] = tat[index] - bt[index];
    }
}
}

```

```

printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
}

return 0;
}

```

Result:

```

C:\Users\nydhi\OneDrive\Doc >
Name: NYDHILE NAGARAJ
USN:1WA24CS195

Enter number of processes: 4
Enter Arrival Time:
0 1 2 3
Enter Burst Time:
8 4 9 5

Process AT      BT      CT      TAT      WT
P1      0      8      17      17      9
P2      1      4      5       4      0
P3      2      9     26     24     15
P4      3      5     10      7      2

Process returned 0 (0x0)   execution time : 10.217 s
Press any key to continue.

```

iii) SJF Preemptive

OUTPUT:

Enter number of processes:

2

Enter AT and BT for P1:

3 4

Enter AT and BT for P2:

5 6

P	AT	BT	CT	TAT	WT
P1	3	4	7	4	0
P2	5	6	13	8	2

Average TAT = 6.00

Average WT = 1.00

000
6-5-21

iii) SJF preemptive

```
#include <stdio.h>
```

```
int main() {
```

```
int n, at[10], bt[10], rt[10], ct[10],  
tat[10], wt[10];
```

```
int i, time = 0, completed = 0, min, index;
```

```
float avg-tat = 0, avg-wt = 0;
```

```
printf("Enter number of processes:");
```

```
scanf("%d", &n);
```



```

for (i=0; i<n; i++) {
    printf("Enter AT & BT for P%d: ", i+1);
    scanf("%d%d", &at[i], &bt[i]);
    rt[i] = bt[i];
}

```

```

while (completed < n) {
    min = 9999;
    index = -1;
}

```

```

for (i=0; i<n; i++) {
    if (at[i] <= time && at[i] > 0 &&
        rt[i] < min) {
        min = rt[i];
        index = i;
    }
}

```

```

if (index == -1) {
    time++;
} else {
    at[index]--;
    time++;
}

```

```

if (at[index] == 0) {
    ct[index] = time;
    tat[index] = ct[index] - at[index];
    wt[index] = tat[index] - bt[index];
}

```

```

avg_tat += tat[index];
avg_wt += wt[index];
completed++;
}
}
}

```

```

i+1);
printf("\nP\tAT\tBT\tCT\tTAT\n");
for (i=0; i<n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t", i+1, at[i],
        bt[i], ct[i], tat[i], wt[i]);
}

printf("\nAverage TAT = %.2f", avg_tat/n);
printf("\nAverage WT = %.2f\n", avg_wt/n);
return 0;
}

```

OUTPUT

Enter number of processes: 2
 Enter AT and BT for P1:
 3 4
 Enter AT and BT for P2:
 5 6

P	AT	BT	CT	TAT	WT
P1	3	4	7	4	0
P2	5	6	13	8	2

Average TAT = 6.00

Average WT = 1.00

$$\frac{6.00}{6} = 1.00$$

PRIORITY NON-PREEMPTIVE

CODE:

```
#include <stdio.h>

int main()
{
printf("Name: NYDHILE NAGARAJ\nUSN:1WA24CS195\n\n");
    int n, i;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], pr[n], ct[n], tat[n], wt[n];
    int done[n];

    printf("Enter Arrival Time:\n");
    for(i = 0; i < n; i++)
    {
        scanf("%d", &at[i]);
        done[i] = 0;
    }

    printf("Enter Burst Time:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &bt[i]);

    printf("Enter Priority:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &pr[i]);

    int t = 0, completed = 0;
```

```

while(completed < n)
{
    int min = 9999;
    int index = -1;

    for(i = 0; i < n; i++)
    {
        if(at[i] <= t && done[i] == 0)
        {
            if(pr[i] < min)
            {
                min = pr[i];
                index = i;
            }
        }
    }

    if(index == -1)
    {
        t++;
    }
    else
    {
        ct[index] = t + bt[index];
        t = ct[index];

        tat[index] = ct[index] - at[index];
        wt[index] = tat[index] - bt[index];

        done[index] = 1;
        completed++;
    }
}

```

```

    }

    printf("\nProcess\tAT\tBT\tPR\tCT\tTAT\tWT\n");

    for(i = 0; i < n; i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
            i+1, at[i], bt[i], pr[i], ct[i], tat[i], wt[i]);
    }

    return 0;
}

```

Result:

```

C:\Users\nydhi\OneDrive\Doc
Name: NYDHILE NAGARAJ
USN:1WA24CS195

Enter number of processes: 5
Enter Arrival Time:
0 2 3 4 6
Enter Burst Time:
3 2 4 5 1
Enter Priority:
5 3 2 4 1

Process AT      BT      PR      CT      TAT      WT
P1      0       3       5       3       3       0
P2      2       2       3      10       8       6
P3      3       4       2       7       4       0
P4      4       5       4      15      11       6
P5      6       1       1       8       2       1

Process returned 0 (0x0)   execution time : 29.255 s
Press any key to continue.

```

⑥ Priority scheduling non pre-emptive

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    printf("Enter no. of processes: ");
    scanf("%d", &n);

    int pid[n], at[n], bt[n], pr[n];
    int ct[n], tat[n], wt[n];
    bool completed[n];

    for (int i = 0; i < n; i++) {
        printf("Arrival time: ");
        scanf("%d", &at[i]);
        printf("Burst time: ");
        scanf("%d", &bt[i]);
        printf("Priority: ");
        scanf("%d", &pr[i]);
        completed[i] = false;
    }

    int current_time = 0;
    int completed_count = 0;

    while (completed_count < n) {
        int highest_priority = 9999;
```

```

int selected_process = -1;
for(int i=0; i<n; i++) {
    if (at[i] == current_time &&
        completed[i] == false) {
        if (pr[i] < highest_priority) {
            highest_priority = pr[i];
            selected_process = i; }
    }
}

if (selected_process == -1) {
    current_time++; }
else {
    int start_time = current_time;
    ct[selected_process] = start_time +
        bt[selected_process];
    lat[selected_process] = ct[selected_process] -
        at[selected_process];
    wt[selected_process] = lat[selected_process] -
        bt[selected_process];
    current_time = ct[selected_process];
    completed[selected_process] = true;
    completed_count++; }

printf("\n PID \t AT \t BT \t PR \t WT \t TAT \n");
for(int i=0; i<n; i++)
    printf("%d \t %d \t %d \t %d \t %d \t %d \n",
        pid[i], at[i],
        pr[i], wt[i], lat[i]); }

```

return 0; }

OUTPUT

Enter no. of processes : 5

Process	Arrival time	Burst time	Priority
P1	0	3	5
P2	2	2	3
P3	3	5	2
P4	4	4	4
P5	6	1	1

Enter priorities :

5 3 2 4 1

Enter arrival time :

~~0 1 2 3 4 5 6 7 8 9 10~~ 0 2 3 4 6

Enter burst time :

3 2 5 4 1

Process	AT	BT	Priority	CT	TAT	WT	RT
P1	0	3	5	15	15	12	0
P2	2	2	3	9	7	5	0
P3	3	5	2	8	5	0	0
P4	4	4	4	13	9	5	0
P5	6	1	1	7	1	0	0

Average TAT : 7.4

Average WT : 4.4

Be
73-32

PRIORITY PREEMPTIVE

CODE:

```
#include <stdio.h>

int main()
{
printf("Name: NYDHILE NAGARAJ\nUSN:1WA24CS195\n\n");
    int n, i;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], pr[n], rt[n];
    int ct[n], tat[n], wt[n];

    printf("Enter Arrival Time:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &at[i]);

    printf("Enter Burst Time:\n");
    for(i = 0; i < n; i++)
    {
        scanf("%d", &bt[i]);
        rt[i] = bt[i];
    }

    printf("Enter Priority:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &pr[i]);

    int t = 0, completed = 0;
```



```

while(completed < n)
{
    int min = 9999;
    int index = -1;

    for(i = 0; i < n; i++)
    {
        if(at[i] <= t && rt[i] > 0)
        {
            if(pr[i] < min)
            {
                min = pr[i];
                index = i;
            }
        }
    }

    if(index == -1)
    {
        t++;
    }
    else
    {
        rt[index]--;
        t++;

        if(rt[index] == 0)
        {
            ct[index] = t;
            tat[index] = ct[index] - at[index];
            wt[index] = tat[index] - bt[index];

```

```

        completed++;
    }
}

printf("\nProcess\tAT\tBT\tPR\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, at[i], bt[i], pr[i], ct[i], tat[i], wt[i]);
}

return 0;
}

```

Result:

```

C:\Users\nydhi\OneDrive\Doc
Name: NYDHILE NAGARAJ
USN:1WA24CS195

Enter number of processes: 7
Enter Arrival Time:
6 4 1 3 0 9
10
Enter Burst Time:
5 1 2 4 8 6 1
Enter Priority:
6 5 4 4 3 2 1

Process AT    BT    PR    CT    TAT    WT
P1      6      5      6     27     21     16
P2      4      1      5     22     18     17
P3      1      2      4     17     16     14
P4      3      4      4     21     18     14
P5      0      8      3      8      8      0
P6      9      6      2     16      7      1
P7     10      1      1     11      1      0

Process returned 0 (0x0)   execution time : 42.023 s
Press any key to continue.

```

7. Priority scheduling preemptive

```
#include <stdio.h>
int main() {
    int n;
    printf("Enter no. of processes: ");
    scanf("%d", &n);
    int pid[n], at[n], bt[n], pr[n];
    int rt[n], ct[n], tat[n], wt[n];
```

```
for (int i=0; i<n; i++) {
    printf("Arrival time: ");
    scanf("%d", &at[i]);
    printf("Burst time: ");
    scanf("%d", &bt[i]);
    printf("priority: ");
    scanf("%d", &pr[i]);
    rt[i] = bt[i]; }
```

```
int current_time = 0;
int completed = 0;
```

```
while (completed < n) {
    int highest_priority = 9999;
    int selected_process = -1;
```

```
for (int i=0; i<n; i++) {
    if (at[i] <= current_time && rt[i] > 0)
        if (pr[i] < highest_priority) {
```

```

highest-priority = p[i];
selected-process = i; } }
}

if (selected-process == -1) {
    current-time++; }
else {
    at[selected-process]--;
    current-time++;
    if (at[selected-process] == 0) {
        ct[selected-process] = current-time;
        tat[selected-process] = ct [selected-process]
            - at[selected-process];
        wt[selected-process] = tat[selected-process]
            - bt[selected-process];
        completed++; } }
}

printf("\n PID \t AT \t BT \t PR \t WT \t TAT");

for (int i=0; i<n; i++) {
    printf("%d \t %d \t %d \t %d \t %d \t %d",
        pid[i], at[i], bt[i], pr[i], wt[i], tat[i]);
    }
return 0;
}

```

OUTPUT :

Enter no. of processes : 7

Enter priorities :

3 4 4 5 2 6 1

Enter arrival times :

0 1 3 4 5 6 10

Enter burst times :

8 2 4 1 6 5 1

PID	Priority	AT	BT	CT	TAT	WT	R ⁿ
P6	6	6	5	27	21	16	14
P4	5	4	1	22	18	17	17
P2	4	1	2	17	16	14	1
P3	4	3	4	21	18	14	1
P1	3	0	2	15	15	7	
P5	2	5	6	12	7	1	
P7	1	10	1	11	1	0	

Average TAT : 13.714286
Average WT : 9.857143

6
17-3-2

ROUND ROBIN

CODE:

```
#include <stdio.h>

int main()
{
printf("Name: NYDHILE NAGARAJ\nUSN:1WA24CS195\n\n");
    int n, tq, i;

    printf("Enter number of processes: ");
    scanf("%d",&n);

    printf("Enter Time Quantum: ");
    scanf("%d",&tq);

    int pid[n], at[n], bt[n], rt[n];
    int wt[n], tat[n], ct[n], resp[n];
    int queue[100];
    int front=0, rear=0;

    for(i=0;i<n;i++)
    {
        printf("Enter Process ID, Arrival Time, Burst Time:\n");
        scanf("%d %d %d",&pid[i],&at[i],&bt[i]);

        rt[i] = bt[i];
        wt[i] = 0;
        tat[i] = 0;
        resp[i] = -1;
    }
```

```

int current_time = 0;
int completed = 0;
int visited[n];

for(i=0;i<n;i++)
    visited[i] = 0;

while(completed < n)
{
    for(i=0;i<n;i++)
    {
        if(at[i] <= current_time && visited[i] == 0)
        {
            queue[rear++] = i;
            visited[i] = 1;
        }
    }

    if(front == rear)
    {
        current_time++;
        continue;
    }

    int p = queue[front++];

    if(resp[p] == -1)
        resp[p] = current_time - at[p];

    if(rt[p] > tq)
    {
        current_time += tq;
    }
}

```



```

    rt[p] -= tq;

    for(i=0;i<n;i++)
    {
        if(at[i] <= current_time && visited[i] == 0)
        {
            queue[rear++] = i;
            visited[i] = 1;
        }
    }

    queue[rear++] = p;
}
else
{
    current_time += rt[p];
    rt[p] = 0;

    ct[p] = current_time;
    tat[p] = ct[p] - at[p];
    wt[p] = tat[p] - bt[p];
    completed++;
}
}

float avg_wt=0, avg_tat=0;

printf("\nPID\tAT\tBT\tCT\tTAT\tWT\tRT\n");

for(i=0;i<n;i++)
{
    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",

```

```

pid[i],at[i],bt[i],ct[i],tat[i],wt[i],resp[i]);

avg_wt += wt[i];
avg_tat += tat[i];
}

printf("\nAverage Waiting Time = %.2f",avg_wt/n);
printf("\nAverage Turnaround Time = %.2f",avg_tat/n);

return 0;
}

```

Result:

```

Name: NYDHILE NAGARAJ
USN:1WA24CS195

Enter number of processes: 5
Enter Time Quantum: 2
Enter Process ID, Arrival Time, Burst Time:
1 0 5
Enter Process ID, Arrival Time, Burst Time:
2 1 3
Enter Process ID, Arrival Time, Burst Time:
2 3 1
Enter Process ID, Arrival Time, Burst Time:
4 3 2
Enter Process ID, Arrival Time, Burst Time:
5 4 3

PID    AT    BT    CT    TAT    WT    RT
1       0     5    13     13     8     0
2       1     3    12     11     8     1
2       3     1     7      4     3     3
4       3     2     9      6     4     4
5       4     3    14     10     7     5

Average Waiting Time = 6.00
Average Turnaround Time = 8.80
Process returned 0 (0x0)   execution time : 28.777 s
Press any key to continue.

```

(8)

Round Robin

```
#include <stdio.h>
int main() {
    int n, tq;
    printf("Enter no. of processes: ");
    scanf("%d", &n);
    printf("Enter time quantum);
    scanf("%d", &tq);
    int pid[n], at[n], bt[n], rt[n];
    int wt[n], tat[n], ct[n];
```

```
for(int i=0; i<n; i++) {
    printf("process & arrival time: ");
    scanf("%d", &at[i]);
    printf("Burst time: ");
    scanf("%d", &bt[i]);
    rt[i] = bt[i];
    wt[i] = 0;
    tat[i] = 0; }
```

```
int time = 0, completed = 0;
int queue[100], front = 0, rear = 0;
int visited[n];
```

```
for(int i=0; i<n; i++)
    visited[i] = 0;
```

```

    at[p] = 0
    completed++;

    for(int i=0; i<n; i++) {
        if (at[i] <= time && visited[i] == 0) {
            queue[rear++] = i;
            visited[i] = 1; } }

    float avg-wt = 0, avg-tat = 0;
    printf("%d\nPID\tLAT\tLBT\tLWT\tTAT\n");

    for(int i=0; i<n; i++) {
        printf("%d\t%d\t%d\t%d\t%d\n",
            pid[i], at[i], bt[i], wt[i], tat[i]);
        avg-wt += wt[i];
        avg-tat += tat[i]; }

    printf("Average WT = %.2f", avg-wt/n);
    printf("Average TAT = %.2f\n", avg-tat/n);

    return 0;
}

```

Enter Time quantum: 2
Enter arrival time:
0 1 2 3 4
Enter burst time:
5 3 1 2 3

PID	AT	BT	CT	TAT	WT	RT
1	0	5	13	13	8	0
2	1	3	12	11	8	1
3	2	1	5	3	2	2
4	3	2	9	6	4	4
5	4	3	14	10	7	5

Average TAT: 8.6000
Average WT: 5.8000

13-3-26

Program -2

Question:

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

CODE:

```
#include <stdio.h>

int main()
{
printf("Name: NYDHILE NAGARAJ\nUSN:1WA24CS195\n\n");
    int n, i;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], rt[n], ct[n], tat[n], wt[n], q[n];

    printf("Enter Arrival Time:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &at[i]);

    printf("Enter Burst Time:\n");
    for(i = 0; i < n; i++)
    {
        scanf("%d", &bt[i]);
        rt[i] = bt[i];
    }

    printf("Enter Queue (1 = RR, 2 = FCFS):\n");
    for(i = 0; i < n; i++)
```

```

scanf("%d", &q[i]);

int tq;
printf("Enter Time Quantum: ");
scanf("%d", &tq);

int t = 0, completed = 0;

printf("\nGantt Chart:\n| ");

while(completed < n)
{
    int found = 0;

    for(i = 0; i < n; i++)
    {
        if(q[i] == 1 && rt[i] > 0 && at[i] <= t)
        {
            int exec = (rt[i] > tq) ? tq : rt[i];

            for(int k = 0; k < exec; k++)
            {
                printf("P%d ", i+1);
                t++;
            }

            rt[i] -= exec;

            if(rt[i] == 0)
            {
                ct[i] = t;
            }
        }
    }
}

```



```

        completed++;
    }

    found = 1;
}
}

if(!found)
{
    for(i = 0; i < n; i++)
    {
        if(q[i] == 2 && rt[i] > 0 && at[i] <= t)
        {

            while(rt[i] > 0)
            {
                // check if any Q1 arrives → break
                int interrupt = 0;
                for(int j = 0; j < n; j++)
                {
                    if(q[j] == 1 && rt[j] > 0 && at[j] <= t)
                    {
                        interrupt = 1;
                        break;
                    }
                }
            }

            if(interrupt) break;

            printf("P%d ", i+1);
            rt[i]--;

```

```

        t++;
    }

    if(rt[i] == 0)
    {
        ct[i] = t;
        completed++;
    }

    found = 1;
}
}
}

// CPU idle
if(!found)
{
    printf("Idle ");
    t++;
}
}

printf("\n");

// Calculate TAT & WT
for(i = 0; i < n; i++)
{
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
}

printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");

```

```

for(i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
}

return 0;
}

```

Result:

```

C:\Users\nydhil\OneDrive\Doc >
Name: NYDHILE NAGARAJ
USN:1WA24CS195

Enter number of processes: 4
Enter Arrival Time:
0 0 0 10
Enter Burst Time:
4 3 8 5
Enter Queue (1 = RR, 2 = FCFS):
1 1 2 1
Enter Time Quantum: 2

Gantt Chart:
| P1 P1 P2 P2 P1 P1 P2 P3 P3 P3 P4 P4 P4 P4 P4 P3 P3 P3 P3 P3 |

Process AT    BT    CT    TAT   WT
P1      0      4      6      6     2
P2      0      3      7      7     4
P3      0      8     20     20    12
P4     10      5     15      5     0

Process returned 0 (0x0)   execution time : 25.041 s
Press any key to continue

```

17/8/26

Multi Level Queue Scheduling

```
#include <stdio.h>
#define MAX 20
int main() {
    int n, i, j;
    int at[MAX], bt[MAX], queue[MAX];
    int ct[MAX], tat[MAX], wt[MAX], rt[MAX];
    int time = 0, tq = 2;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf
```

Multi Level Queue Scheduling

```
#include <stdio.h>
#define MAX 20
int main() {
    int n, i, time = 0, tq = 2;
    int at[MAX], bt[MAX], rt[MAX], q[MAX];
    int ct[MAX], tat[MAX], wt[MAX];
    int done;
    printf("Enter no. of processes: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("\nProcess %d\n", i+1);
        printf("Enter AT, BT, Queue (1 or 2): ");
        scanf("%d %d %d", &at[i], &bt[i], &q[i]);
```

at[i] = bt[i]; }

printf ("\n --- Execution order --- \n");
int finished = 0;
while (finished < n) {
 int executed = 0;

for (i=0; i<n; i++) {
 if (q[i] == 1 && at[i] > 0 && at[i] <= time) {

executed = 1;
 printf ("Poped", i+1);
 int run = (at[i] > tq) ? tq : at[i];
 time += run;
 at[i] -= run;
 if (at[i] == 0) {
 ct[i] = time;
 finished++; } }
}

if (executed) continue;

int idx = -1;

int min_at = 9999;

for (i=0; i<n; i++) {
 if (q[i] == 2 && at[i] > 0 && at[i] <= time) {

if (at[i] < min_at) {
 min_at = at[i];

idx = i; } }

```

if (idx != -1)
    executed = 1;
printf("P.o.d ", idx + 1);
time += 1;
at[idx] -= 1;
if (at[idx] == 0) {
    ct[idx] = time;
    finished++;
}
if (!executed) {
    time++;
}
for (i = 0; i < n; i++) {
    lat[i] = ct[i] - at[i];
    wt[i] = lat[i] - bt[i];
}

printf("\n\nPID \t AT \t BT \t D \t CT  

\t AT \t WT \n");
for (i = 0; i < n; i++) {
    printf("P.o.d \t o.p.d \t o.p.d \t o.p.d  

\t o.p.d \t o.p.d \t o.p.d \n",
        i + 1, at[i], bt[i], q[i],
        ct[i], lat[i], wt[i]);
}
return 0;
}

```


OUTPUT

Enter number of processes : 4

Process 1

Enter AT, BT, Queue (1 or 2) : 0 4 1

Process 2

Enter AT, BT, Queue (1 or 2) : 0 3 1

Process 3

Enter AT, BT, Queue (1 or 2) : 0 8 2

Process 4

Enter AT, BT, Queue (1 or 2) : 10 5 2

--- Execution order ---

P1 P2 P1 P2 P3 P3 P3 P4 P4 P4
P3 P3 P3 P3 P3

PID	AT	BT	Q	CT	TAT	WT
P1	0	4	1	6	6	2
P2	0	3	1	7	7	4
P3	0	8	2	20	20	12
P4	10	5	1	15	5	0

21/12/20

Program -3

Question:

Write a C program to simulate Real-Time CPU Scheduling algorithms: (Any one)

- a) Rate- Monotonic
- b) Earliest-deadline First
- c) Proportional scheduling

RATE – MONOTONIC

CODE:

```
#include <stdio.h>
#include<math.h>

int main()
{
printf("Name:NYDHILE NAGARAJ\nUSN:1WA24CS195\n\n");
    int n, i, j, t = 0, lcm = 1;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int burst[n], period[n], remaining[n];

    printf("Enter CPU burst times:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &burst[i]);

    printf("Enter time periods:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &period[i]);

    for(i = 0; i < n; i++)
```

```

{
    int max = (lcm > period[i]) ? lcm : period[i];
    while(1)
    {
        if(max % lcm == 0 && max % period[i] == 0)
        {
            lcm = max;
            break;
        }
        max++;
    }
}

```

```

printf("\nLCM=%d\n", lcm);

```

```

printf("\nRate Monotonic Scheduling:\n");

```

```

printf("PID\tBurst\tPeriod\n");

```

```

for(i = 0; i < n; i++)

```

```

    printf("%d\t%d\t%d\n", i+1, burst[i], period[i]);

```

```

float util = 0;

```

```

for(i = 0; i < n; i++)

```

```

    util += (float)burst[i] / period[i];

```

```

float bound = n * (pow(2, (float)1/n) - 1);

```

```

printf("\n%f <= %f => %s\n", util, bound,

```

```

    (util <= bound) ? "true" : "false");

```

```

for(i = 0; i < n; i++)

```

```

    remaining[i] = 0;

printf("\nScheduling occurs for %d ms\n\n", lcm);

for(t = 0; t < lcm; t++)
{

    for(i = 0; i < n; i++)
    {
        if(t % period[i] == 0)
            remaining[i] = burst[i];
    }

    int min = -1;
    for(i = 0; i < n; i++)
    {
        if(remaining[i] > 0)
        {
            if(min == -1 || period[i] < period[min])
                min = i;
        }
    }

    if(min != -1)
    {
        printf("%dms onwards: Process %d running\n", t, min+1);
        remaining[min]--;
    }
    else
    {

```

```

        printf("%dms onwards: CPU is idle\n", t);
    }
}
return 0;
}

```

Result:

```

C:\Users\nydhil\OneDrive\Doc
Name: NYDHILE NAGARAJ
USN: 1WA24CS195

Enter number of processes: 2
Enter CPU burst times:
20 35
Enter time periods:
50 100

LCM=100

Rate Monotonic Scheduling:
PID    Burst   Period
1       20       50
2       35       100

0.750000 <= 0.828427 => true

Scheduling occurs for 100 ms

0ms onwards: Process 1 running
1ms onwards: Process 1 running
2ms onwards: Process 1 running
3ms onwards: Process 1 running
4ms onwards: Process 1 running
5ms onwards: Process 1 running

```

LAB PROGRAM 3

i) RMS

```
#include <stdio.h>
#define MAX 10
```

```
int gcd(int a, int b) {
    if (b == 0) return a;
    return gcd(b, a % b);
}
```

```
int lcm(int a, int b) {
    return (a * b) / gcd(a, b);
}
```

```
int findLcm(int arr[], int n) {
    int result = arr[0];
    for (int i = 1; i < n; i++) {
        result = lcm(result, arr[i]);
    }
    return result;
}
```

```
int main() {
    int n;
    int burst[MAX], period[MAX];
    int remaining[MAX];
    int current_time = 0;
```

```
printf("Enter the no. of processes: ");
scanf("%d", &n);
```

classmate
Date _____
Page _____

```
printf("Enter the CPU burst time : \n");
for(int i=0; i<n; i++) {
    scanf("%d", &burst[i]);
}
```

```
printf("Enter the time periods : \n");
for(int i=0; i<n; i++) {
    scanf("%d", &period[i]);
}
```

```
int hyperperiod = find_lcm(period, n);
printf("LCM = %d \n \n", hyperperiod);
```

```
printf("Rate Monotone Scheduling : \n");
printf("PID \t Burst \t Period \n");
for(int i=0; i<n; i++) {
    printf("%d \t %d \t %d \n",
        i+1, burst[i], period[i]);
}
```

```
float utilization = 0;
for(int i=0; i<n; i++) {
    utilization += (float) burst[i] / period[i];
}
```

```
float bound = n * pow(2, 1.0/n) - 1;
```


classmate
Date _____
Page _____

```
printf("\n %f <= %f ==> %f \n",
utilization, bound, (utilization <= bound)
?"True" : "False");
```

```
printf("scheduling occurs for %d ms
in %d", hyperperiod);
```

```
for(int i=0; i<n; i++) {
    remaining[i] = 0; }
```

```
while (current time < hyperperiod) {
    for (int i=0; i<n; i++) {
        if (current time % period[i] == 0) {
            remaining[i] = burst[i];
        }
    }
}
```

```
int selected = -1;
int min-period = 9999;
```

```
for (int i=0; i<n; i++) {
    if (remaining[i] > 0 && period[i] <
        min-period) {
        min-period = period[i];
        selected = i;
    }
}
```


classmate
Date _____
Page _____

```

if (selected != -1) {
    printf ("%dms onwards: Process %d\n",
            current_time, selected + 1);
    remaining[selected]--;
}
else {
    printf ("%dms onwards: CPU is
    idle\n", current_time);
}

current_time++;
}
return 0;
}

```

OUTPUT

Enter the number of processes: 2

Enter the CPU burst times:

20 35

Enter the time periods:

50 100

LCM = 100

Rate Monotone Scheduling:

PID	Burst	Period
1	20	50
2	35	100

$0.750000 \leq 0.828427 \Rightarrow \text{True}$
Scheduling occurs for 100 ms

EARLIEST DEADLINE FIRST

CODE:

```
#include <stdio.h>
#include <limits.h>

int main() {
    printf("Name: NYDHILE NAGARAJ\nUSN:1WA24CS195\n\n");
    int n, i, time, hyperperiod = 20;

    printf("Enter number of tasks: ");
    scanf("%d", &n);

    int C[n], D[n], T[n];
    int remaining[n];
    int next_arrival[n];
    int abs_deadline[n];

    for(i = 0; i < n; i++) {
        printf("Enter Capacity, Deadline, Period for Task %d: ", i + 1);
        scanf("%d %d %d", &C[i], &D[i], &T[i]);

        remaining[i] = 0;
        next_arrival[i] = 0;
        abs_deadline[i] = 0;
    }
```

```

printf("\n--- EDF Scheduling (Periodic Tasks) ---\n");

for(time = 0; time < hyperperiod; time++) {

    for(i = 0; i < n; i++) {
        if(time == next_arrival[i]) {
            remaining[i] = C[i];
            abs_deadline[i] = time + D[i];
            next_arrival[i] += T[i];
        }
    }

    int selected = -1;
    int min_deadline = INT_MAX;

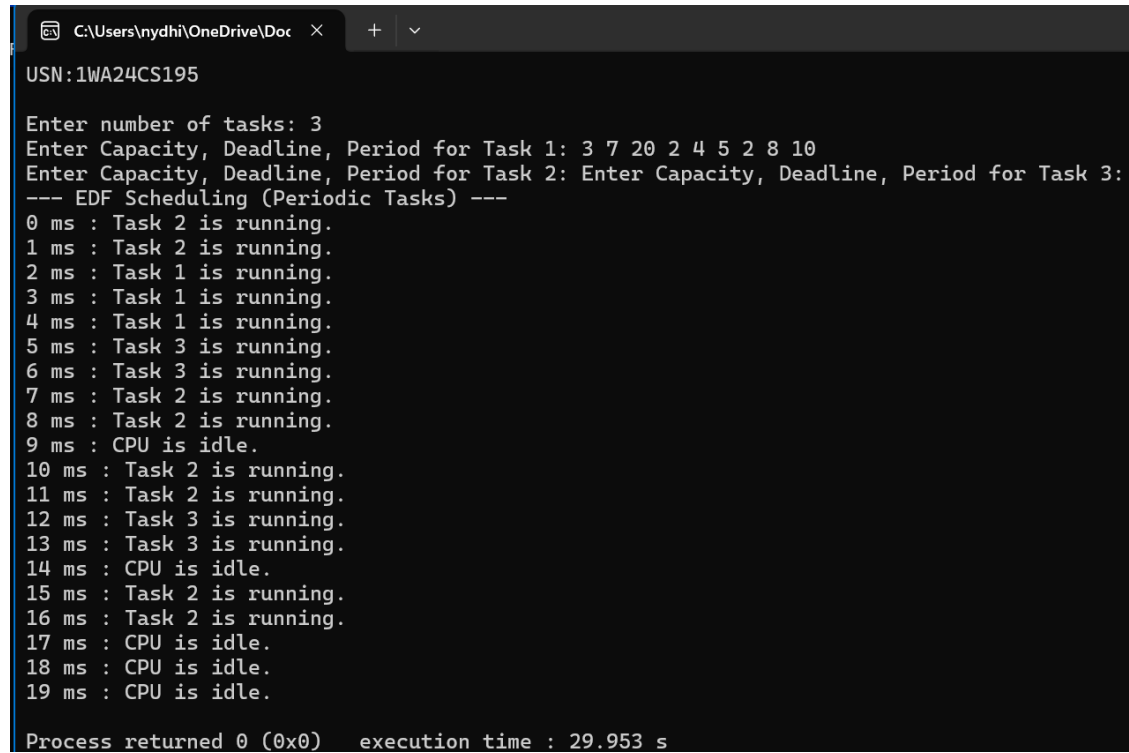
    for(i = 0; i < n; i++) {
        if(remaining[i] > 0 && abs_deadline[i] < min_deadline) {
            min_deadline = abs_deadline[i];
            selected = i;
        }
    }

    if(selected == -1) {
        printf("%d ms : CPU is idle.\n", time);
    } else {
        printf("%d ms : Task %d is running.\n", time, selected + 1);
        remaining[selected]--;
    }
}

```

```
    return 0;
}
```

Result:



```
C:\Users\nydh\OneDrive\Doc x + v
USN:1WA24CS195
Enter number of tasks: 3
Enter Capacity, Deadline, Period for Task 1: 3 7 20 2 4 5 2 8 10
Enter Capacity, Deadline, Period for Task 2: Enter Capacity, Deadline, Period for Task 3:
--- EDF Scheduling (Periodic Tasks) ---
0 ms : Task 2 is running.
1 ms : Task 2 is running.
2 ms : Task 1 is running.
3 ms : Task 1 is running.
4 ms : Task 1 is running.
5 ms : Task 3 is running.
6 ms : Task 3 is running.
7 ms : Task 2 is running.
8 ms : Task 2 is running.
9 ms : CPU is idle.
10 ms : Task 2 is running.
11 ms : Task 2 is running.
12 ms : Task 3 is running.
13 ms : Task 3 is running.
14 ms : CPU is idle.
15 ms : Task 2 is running.
16 ms : Task 2 is running.
17 ms : CPU is idle.
18 ms : CPU is idle.
19 ms : CPU is idle.
Process returned 0 (0x0)   execution time : 29.953 s
```

ii) EDF

```
#include <stdio.h>
#define MAX 10
```

```
int gcd(int a, int b) {
    if (b == 0) return a;
    return gcd(b, a % b);
}
```

```
int lcm(int a, int b) {
    return (a * b) / gcd(a, b);
}
```

```
int find_lcm(int arr[], int n) {
    int result = arr[0];
    for (int i = 1; i < n; i++) {
```

ii) EDF

```
#include <stdio.h>
#define MAX 10
#define INF 9999
```

```
int gcd(int a, int b) {
    return b ? gcd(b, a % b) : a;
}
```

```
int lcm(int a, int b) {
    return a * b / gcd(a, b);
}
```


mate

classmate
Date _____
Page _____

```

int main () {
    int n, i, t, hp;
    int b[MAX], p[MAX], d[MAX],
        rem[MAX], wl[MAX];

    printf("Enter number of process :");
    scanf("%d", &n);

    for (i=0; i<n; i++) {
        printf("Enter burst, deadline, period\nfor T%d : ", i+1);
        scanf("%d %d %d", &b[i], &d[i],
            &p[i]);
        rem[i] = 0;
    }

    hp = p[0];
    for (i=1; i<n; i++)
        hp = *min(h, p[i]);

    printf("\nPID\tBurst\tDeadline\tPeriod\n");
    for (i=0; i<n; i++)
        printf("\n Scheduling\n");
    printf("%d\t%d\t%d\t%d\t%d\n",
        i+1, b[i], d[i], p[i]);
    printf("\n scheduling occurs for %d\nms\n", hp);
}

```



```

for (t=0; t<bp; t++) {
    for (i=0; i<n; i++)
        if (t/opp[i] == 0) {
            rem[i] = b[i];
            ad[i] = t+d[i];
        }
    int sel = -1, min = INF;
    for (i=0; i<n; i++)
        if (rem[i]>0 && ad[i]<min) {
            min = ad[i];
            sel = i;
        }
    if (sel == -1)
        printf ("%dms: CPU is idle.\n", t);
    else {
        printf ("%dms: Task %d is running\n", t, sel+1);
        rem[sel]--;
    }
}

```

OUTPUT

Enter no. of processes: 3 2 3 2

Enter CPU burst times: ~~20 20 20~~

Enter time periods: ~~20 20 20~~ 5 20 10

PID	Burst	Deadline	Period
1	2	4	5
2	3	7	20
3	2	8	10

Due
28/5/22

PROPORTIONAL SHARE SCHEDULING

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

typedef struct {
    char name[5];
    int tickets;
} Process;

int main() {
    printf("Name: NYDHILE NAGARAJ\nUSN:1WA24CS195\n\n");
    int n, total_tickets = 0;
    float total_T = 0.0;

    printf("Enter the number of Processes: ");
    scanf("%d", &n);
    Process p[n];
    srand(time(NULL));

    for (int i = 0; i < n; i++) {
        printf("\nProcess %d:\n", i + 1);
        sprintf(p[i].name, "P%d", i + 1);
        printf("Tickets: ");
        scanf("%d", &p[i].tickets);
        total_tickets += p[i].tickets;
        total_T += p[i].tickets;
    }
```

```

printf("\n--- Proportional Share Scheduling ---\n");
printf("Enter the Time Period for scheduling: ");
int m;
scanf("%d", &m);

for (int i = 0; i < m; i++) {
    int winning_ticket = rand() % total_tickets + 1;
    int accumulated_tickets = 0;
    int winner_index;

    for (int j = 0; j < n; j++) {
        accumulated_tickets += p[j].tickets;
        if (winning_ticket <= accumulated_tickets) {
            winner_index = j;
            break;
        }
    }

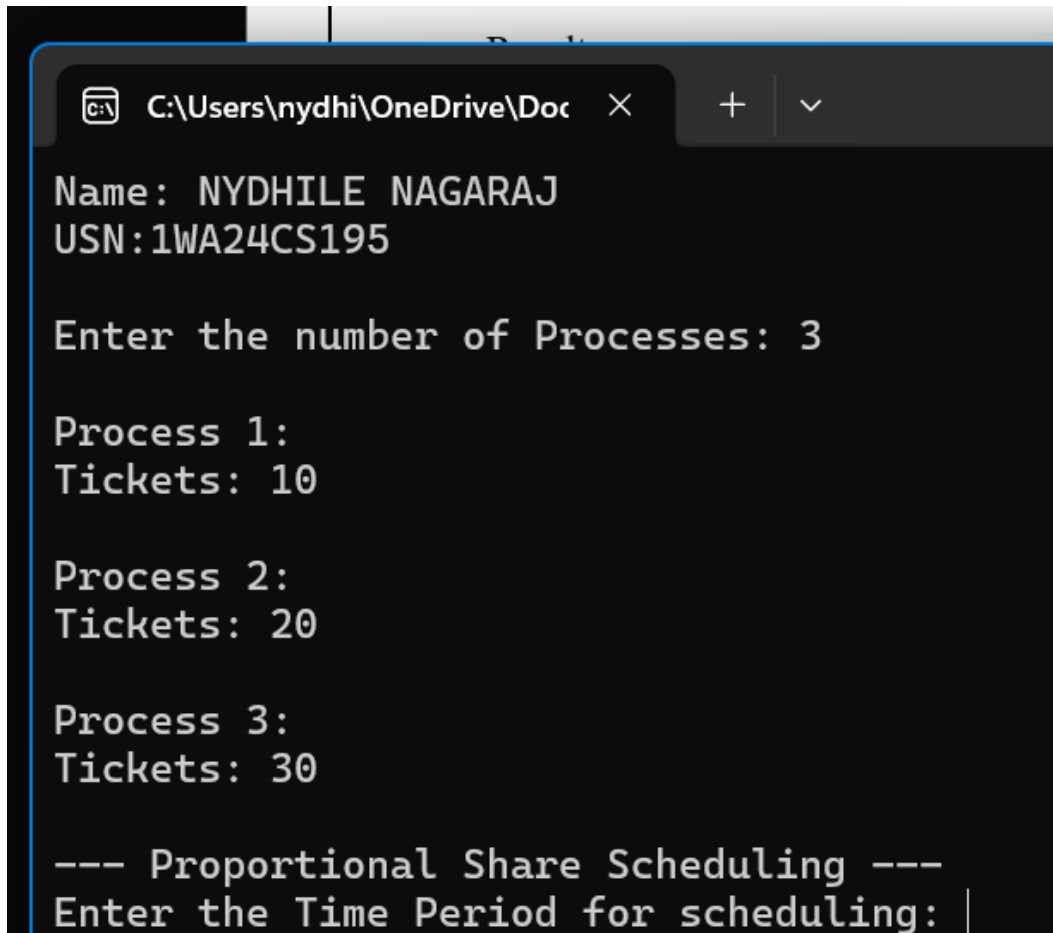
    printf("Tickets picked: %d, Winner: %s\n", winning_ticket,
        p[winner_index].name);
}

for (int i = 0; i < n; i++) {
    printf("\nThe Process: %s gets %0.2f%% of Processor Time.\n", p[i].name,
        ((p[i].tickets / total_T) * 100));
}

return 0;
}

```

Result:



```
C:\Users\nydhi\OneDrive\Doc X + v
Name: NYDHILE NAGARAJ
USN:1WA24CS195

Enter the number of Processes: 3

Process 1:
Tickets: 10

Process 2:
Tickets: 20

Process 3:
Tickets: 30

--- Proportional Share Scheduling ---
Enter the Time Period for scheduling: |
```

iii) proportional share scheduling

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
typedef struct {
    char name[50]; int tickets;
}
process;

int main() {
    int n, total_tickets = 0; float total_T = 0.0;
    printf("Enter the number of processes: ");
    scanf("%d", &n);

    process p[n];
    srand(time(NULL));

    for(int i=0; i<n; i++) {
        printf("\nProcess o/p: \n", i+1);
        printf("p[i].name, \"P%d\", i+1);
        printf("tickets: ");
        scanf("%d", &p[i].tickets);
        total_tickets += p[i].tickets;
        total_T += p[i].tickets;
    }

```



```

    printf("\n--- proportional share  
scheduling --- \n");
    printf("Enter the time period for  
scheduling: ");
    int m;

    scanf("%d", &m);
    for (int i = 0; i < m; i++) {
        int winning_ticket = rand() %  
total_tickets + 1;
        int accumulated_tickets = 0;
        int winner_index;

        for (int j = 0; j < n; j++) {
            accumulated_tickets += p[j].tickets;
            if (winning_ticket <= accumulated_tickets) {
                winner_index = j;
                break;
            }
        }

        printf("Tickets picked: %d,  
winner: %s \n", winning_ticket,  
p[winner_index].name);
    }

    for (int i = 0; i < n; i++) {
        printf("\n the process: %s gets  
%0.2f %%% of processor time. \n");
    }

```

classmate
Date _____
Page _____

```

p[i].name, ((p[i].tickets/total T)
* 100));
}
return 0; }

```

OUTPUT :

Enter the no. of processes : 3
 Process 1 :
 Tickets : 10
 Process 2 :
 Tickets : 20
 Process 3 :
 Tickets : 30

--- Proportional share scheduling ---

Enter the time period : 5

Tickets picked :	7	winner :	P1
"	46	"	P3
"	24	"	P2
"	37	"	P3
"	44	"	P3

The process : P1 gets 16.67%

The process : P2 gets 33.33%

The process : P3 gets 50.00%

lb
20/1/20

Program -4

Question:

Write a C program to simulate: (Any one)

- a) Producer-Consumer problem using semaphores.
- b) Dining-Philosopher's problem

PRODUCER – CONSUMER

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <semaphore.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int top = -1;
int item = 0;

sem_t empty, full, mutex;

void produce() {
    sem_wait(&empty);
    sem_wait(&mutex);

    item++;
    top++;
    buffer[top] = item;

    printf("Producer has produced: Item %d\n", item);
```

```

    sem_post(&mutex);
    sem_post(&full);
}

void consume() {
    sem_wait(&full);
    sem_wait(&mutex);

    int consumed = buffer[top];
    printf("Consumer has consumed: Item %d\n", consumed);

    top--;

    sem_post(&mutex);
    sem_post(&empty);
}

int main() {
    printf("Name: NYDHILE NAGARAJ\nUSN:1WA24CS195\n\n");
    int choice;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    sem_init(&mutex, 0, 1);

    while (1) {
        printf("\nEnter 1. Producer 2. Consumer 3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:

```

```
    if (top == BUFFER_SIZE - 1) {
        printf("Buffer is full!\n");
    } else {
        produce();
    }
    break;

case 2:
    if (top == -1) {
        printf("Buffer is empty!\n");
    } else {
        consume();
    }
    break;

case 3:
    printf("Exiting...\n");
    exit(0);

default:
    printf("Invalid choice!\n");
}
}

return 0;
}
```

Result:

```
C:\Users\nydhi\OneDrive\Doc X + v
USN:1WA24CS195

Enter 1. Producer 2. Consumer 3. Exit
Enter your choice: 1
Producer has produced: Item 1

Enter 1. Producer 2. Consumer 3. Exit
Enter your choice: 1
Producer has produced: Item 2

Enter 1. Producer 2. Consumer 3. Exit
Enter your choice: 1
Producer has produced: Item 3

Enter 1. Producer 2. Consumer 3. Exit
Enter your choice: 2
Consumer has consumed: Item 3

Enter 1. Producer 2. Consumer 3. Exit
Enter your choice: 2
Consumer has consumed: Item 2

Enter 1. Producer 2. Consumer 3. Exit
Enter your choice: 3
Exiting...

Process returned 0 (0x0)   execution time : 18.359 s
Press any key to continue.
```

Q) Write a C program to simulate Producer Consumer problem using semaphores

```
#include <stdio.h>
#include <stdlib.h>
#include <semaphore.h>
#define SIZE 5
int stack[SIZE];
int top = -1;
sem_t empty, full, mutex;

void produce(int item) {
    sem_wait(&empty);
    sem_wait(&mutex);
    top++;
    stack[top] = item;
    printf("producer has produced : \n", item);
    sem_post(&mutex);
    sem_post(&full);
}

void consume() {
    sem_wait(&full);
    sem_wait(&mutex);
    int item = stack[top];
    top--;
    printf("consumer has consumed : \n", item);
    sem_post(&mutex);
    sem_post(&empty);
}
```

```

sem_post(&mutex);
sem_post(empty); }

int main() {
    int choice, item;
    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);
    sem_init(&mutex, 0, 1);

    while(1) {
        printf("\n1. Producer\n2. Consumer\n3. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        switch(choice) {
            case 1:
                if (top == SIZE-1) {
                    printf("Buffer is full\n"); }
                else {
                    printf("Enter item: ");
                    scanf("%d", &item);
                    produce(item); }
                break;
            case 2:
                if (top == -1) {
                    printf("Buffer is empty!\n"); }
                else { consume(); }
        }
    }
}

```


24/4/26

case 3:

```
printf("Exiting... \n");  
exit(0);
```

default:

```
printf("Exiting... \n");  
* printf("Invalid choice! \n");  
}  
}
```

```
return 0;
```

```
}
```

OUTPUT:

1. Producer

2. Consumer

3. Exit

Enter choice : 2

Buffer is empty!

1. Producer

2. Consumer

3. Exit

Enter choice : 1

Enter item : 1

Producer has produced : Item 1

1. Producer

2. Consumer

3. Exit

Enter choice : 1

Enter item : 3 Producer has produced : Item 3

DINING – PHILOSOPHER’S

CODE:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define N 5

pthread_mutex_t forks[N];
pthread_t philosophers[N];

void* philosopher(void* num) {
    int id = *(int*)num;
    int left = id;
    int right = (id + 1) % N;

    for (int i = 0; i < 1; i++) {
        printf("Philosopher %d is thinking.\n", id);
        sleep(1);
        if (id % 2 == 0) {
            pthread_mutex_lock(&forks[left]);
            printf("Philosopher %d picked left fork %d\n", id, left);

            pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked right fork %d\n", id, right);
        } else {
            pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked right fork %d\n", id, right);

            pthread_mutex_lock(&forks[left]); // FIXED TYPO HERE
        }
    }
}
```

```

        printf("Philosopher %d picked left fork %d\n", id, left);
    }

    printf("Philosopher %d is eating.\n", id);
    sleep(2);

    pthread_mutex_unlock(&forks[left]);
    pthread_mutex_unlock(&forks[right]);

    printf("Philosopher %d put down forks %d and %d\n", id, left, right);
}

return NULL;
}

int main() {
    printf("Name: NYDHILE NAGARAJ\nUSN:1WA24CS195\n\n");
    int ids[N];

    for (int i = 0; i < N; i++) {
        pthread_mutex_init(&forks[i], NULL);
        ids[i] = i;
    }

    for (int i = 0; i < N; i++) {
        pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
    }

    for (int i = 0; i < N; i++) {
        pthread_join(philosophers[i], NULL);
    }
}

```

```

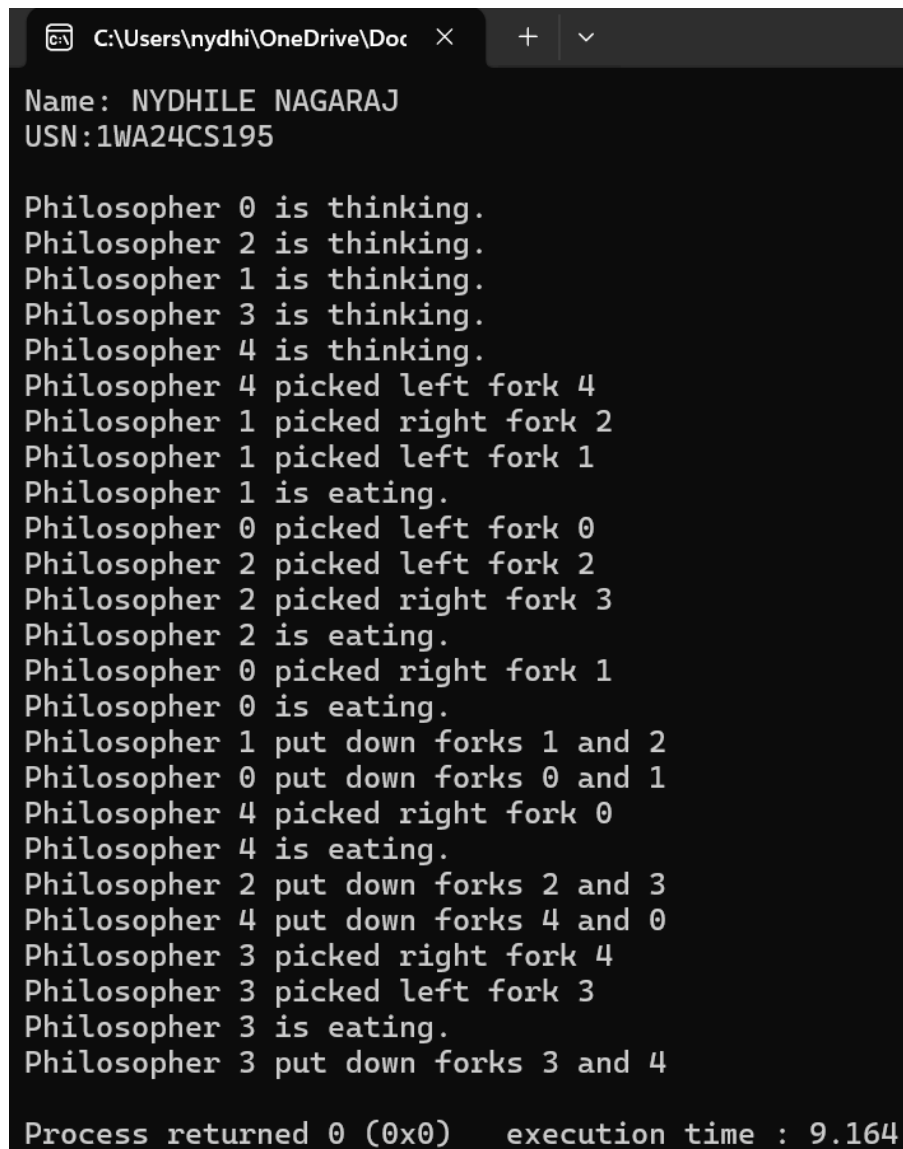
    }

    for (int i = 0; i < N; i++) {
        pthread_mutex_destroy(&forks[i]);
    }

    return 0;
}

```

Result:



```

Name: NYDHILE NAGARAJ
USN:1WA24CS195

Philosopher 0 is thinking.
Philosopher 2 is thinking.
Philosopher 1 is thinking.
Philosopher 3 is thinking.
Philosopher 4 is thinking.
Philosopher 4 picked left fork 4
Philosopher 1 picked right fork 2
Philosopher 1 picked left fork 1
Philosopher 1 is eating.
Philosopher 0 picked left fork 0
Philosopher 2 picked left fork 2
Philosopher 2 picked right fork 3
Philosopher 2 is eating.
Philosopher 0 picked right fork 1
Philosopher 0 is eating.
Philosopher 1 put down forks 1 and 2
Philosopher 0 put down forks 0 and 1
Philosopher 4 picked right fork 0
Philosopher 4 is eating.
Philosopher 2 put down forks 2 and 3
Philosopher 4 put down forks 4 and 0
Philosopher 3 picked right fork 4
Philosopher 3 picked left fork 3
Philosopher 3 is eating.
Philosopher 3 put down forks 3 and 4

Process returned 0 (0x0)    execution time : 9.164

```

P1 takes ~~S2~~ S2

P1 → takes ~~many~~ S1 → eats → releases S2, S1

P0 takes S0

P0 takes S1 → eats → releases S0, S1

P4 takes S4

P4 takes S0 → eats → releases S4, S0

so order is

P3 → P2 → P1 → P0 → P4

◦ code: dining philosophers using ~~the~~ pthreads

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
#include <unistd.h>
```

```
#define N 5
```

```
pthread_mutex_t forks[N];
```

```
pthread_t philosophers[N];
```

```
void * philosopher(void * num) {
```

```
    int id = *(int *) num;
```

```
    int left = id;
```

```
    int right = (id + 1) % N;
```

```
    while (1) {
```

```
        printf("philosopher %d is thinking.\n", id);
```



```

sleep(1);
if (id % 2 == 0) {
    pthread_mutex_lock(&forks[left]);
    printf("philosopher %d picked left fork\n", id, left);
    pthread_mutex_lock(&forks[right]);
    printf("philosopher %d picked right fork\n", id, right);
}

else {
    pthread_mutex_lock(&forks[right]);
    printf("philosopher %d picked right fork\n", id, right);
    pthread_mutex_lock(&forks[left]);
    printf("philosopher %d picked left fork\n", id, left);
}

printf("philosopher %d is eating\n", id);
sleep(2);

pthread_mutex_unlock(&forks[left]);
pthread_mutex_unlock(&forks[right]);

printf("philosopher %d put down\n", id, left, right);
}

return NULL;
}

```



```
int main() {
    int i;
    int ids[N];
    for (i=0; i<N; i++) {
        pthread_mutex_init(&forks[i], NULL);
        ids[i] = i;
    }
    for (i=0; i<N; i++) {
        pthread_create(&philosophers[i], NULL,
            philosopher, &ids[i]);
    }
    for (i=0; i<N; i++) {
        pthread_join(philosophers[i], NULL);
    }
    for (i=0; i<N; i++) {
        pthread_mutex_destroy(&forks[i]);
    }
    return 0;
}
```

OUTPUT :

Philosopher 0 is thinking
Philosopher 1 is thinking
...

Philosopher 0 picked left fork 0
Philosopher 0 picked right fork 1
Philosopher 0 is eating
Philosopher 1 picked right fork 2
...

24-4-26

Program -5

Question:

Write a C program to simulate: (Any one)

- a) Bankers' algorithm for the purpose of deadlock avoidance.
- b) Deadlock Detection

BANKER'S ALGORITHM

CODE:

```
#include <stdio.h>

int main()
{
printf("Name: NYDHILE NAGARAJ\nUSN:1WA24CS195\n\n");
    int n, m, i, j, k;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resources: ");
    scanf("%d", &m);

    int alloc[n][m], max[n][m], need[n][m];
    int avail[m];

    printf("Enter Allocation Matrix:\n");
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < m; j++)
        {
            scanf("%d", &alloc[i][j]);
        }
    }
}
```

```

printf("Enter Maximum Demand Matrix:\n");
for(i = 0; i < n; i++)
{
    for(j = 0; j < m; j++)
    {
        scanf("%d", &max[i][j]);
    }
}

```

```

printf("Enter Available Resources:\n");
for(i = 0; i < m; i++)
{
    scanf("%d", &avail[i]);
}

```

```

// Calculate Need Matrix
for(i = 0; i < n; i++)
{
    for(j = 0; j < m; j++)
    {
        need[i][j] = max[i][j] - alloc[i][j];
    }
}

```

```

int finish[n], safeSeq[n];
int work[m];

```

```

for(i = 0; i < n; i++)
    finish[i] = 0;

```

```

for(i = 0; i < m; i++)

```

```

    work[i] = avail[i];

int count = 0;

while(count < n)
{
    int found = 0;

    for(i = 0; i < n; i++)
    {
        if(finish[i] == 0)
        {
            int possible = 1;

            for(j = 0; j < m; j++)
            {
                if(need[i][j] > work[j])
                {
                    possible = 0;
                    break;
                }
            }

            if(possible)
            {
                for(k = 0; k < m; k++)
                {
                    work[k] += alloc[i][k];
                }

                safeSeq[count] = i;
                count++;
            }
        }
    }
}

```

```

        finish[i] = 1;
        found = 1;
    }
}

if(found == 0)
{
    printf("System is not in a safe state.\n");
    return 0;
}

printf("\nSystem is in a safe state.\n");
printf("Safe sequence is: ");

for(i = 0; i < n; i++)
{
    printf("P%d", safeSeq[i]);

    if(i != n - 1)
        printf(" -> ");
}

printf("\n");

return 0;
}

```

Result:

```
C:\Users\nydhi\OneDrive\Doc  X + v
Name: NYDHILE NAGARAJ
USN:1WA24CS195

Enter number of processes: 5
Enter number of resources: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 2 1
2 1 1
0 0 2
Enter Maximum Demand Matrix:
7 5 3
3 2 2
7 8 9
1 3 4
5 6 7
Enter Available Resources:
3 3 2
System is not in a safe state.

Process returned 0 (0x0)   execution time : 50.951 s
Press any key to continue.
```


Bankers Algorithm

- 1) Safety Algorithm
 - i) Let work and finish be vectors of length m & n
 initialize work = available
 finish[i] = false for $i = 0$ to $n-1$
 2. Find an index i such that
 - a) finish[i] = false
 - b) need[i] ≤ work
 3. work = work + allocation
 finish[i] = true
 go to step 2
 4. If finish[i] = true for all i , then system is in safe state

Ex:

Process	Allocation	Max	Available	Used
P ₀	0110	0210	1520	0100
P ₁	1231	1652		0421
P ₂	1365	2366		1001
P ₃	0632	0652		0020
P ₄	0014	0656		0642

CODE:

```
#include <stdio.h>

int main() {
    int n, m, i, j, k;
    printf("Enter number of processes");
    scanf("%d", &n);
    printf("Enter number of resources:");
    scanf("%d", &m);
```

classmate
Date _____
Page _____

```

int alloc[n][m], max[n][m], need[n][m];
int avail[m], finish[n], safe[n];
printf("Enter allocation matrix: \n");
for (i=0; i<n; i++) {
    for (j=0; j<m; j++) {
        scanf("%d", &alloc[i][j]);
    }
}

```

```

printf("Enter maximum demand matrix);
for (i=0; i<n; i++) {
    for (j=0; j<m; j++) {
        scanf("%d", &max[i][j]);
    }
}

```

```

printf("Enter available resources: ");
for (i=0; i<m; i++) {
    scanf("%d", &avail[i]);
}
for (i=0; i<n; i++) {
    for (j=0; j<m; j++) {
        need[i][j] = max[i][j] - allocation[i][j];
    }
}

```

```

for (i=0; i<n; i++) {
    finish[i] = 0;
}

```

```

int count = 0;
while (count < n) {
    int found = 0;
    for (i=0; i<n; i++) {
        if (finish[i] == 0) {
            for (j=0; j<m; j++) {
                if (need[i][j] > avail[j])
                    break;
            }
        }
    }
}

```

```

if (j == m) {
    for (k = 0; k < m; k++) {
        avail[k] += alloc[i][k];
    }
    safseq[count++] = i;
    finish[i] = 1;
    found = 1; } } }

```

```

if (found == 0) {
    printf("System not in safe state");
    return 0; } }

```

```

printf("\n System is in a safe state\n");
printf("safe sequence is ");
for (i = 0; i < n; i++) {
    printf("P%d", safseq[i]);
    if (i != n - 1)
        printf(" -> "); }
return 0;
}

```

OUTPUT

Enter no. of resources: 5

Enter no. of resources: 4

Enter allocation matrix:

0 1 1 0

1 2 3 1

1 3 6 5

0 6 3 2

0 0 1 4

Enter maximum demand matrix

0 2 1 0

1 6 5 2

2 3 6 6

0 6 5 2

0 6 5 6

Enter available resources:

1 5 2 0

System is in a safe state.

Safe sequence is: $P_0 \rightarrow P_3 \rightarrow P_4 \rightarrow P_1 \rightarrow P_2$

DEADLOCK DETECTION

CODE:

```
#include <stdio.h>

int main()
{
printf("Name: NYDHILE NAGARAJ\nUSN:1WA24CS195\n\n");
    int n, m, i, j, k;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resources: ");
    scanf("%d", &m);

    int alloc[n][m], req[n][m];
    int avail[m], finish[n], safe[n];

    printf("Enter allocation matrix:\n");
    for(i = 0; i < n; i++)
    {
        printf("Process %d: ", i);
        for(j = 0; j < m; j++)
            scanf("%d", &alloc[i][j]);
    }

    printf("Enter request matrix:\n");
    for(i = 0; i < n; i++)
    {
        printf("Process %d: ", i);
        for(j = 0; j < m; j++)
```

```

        scanf("%d", &req[i][j]);
    }

    printf("Enter available resources: ");
    for(i = 0; i < m; i++)
        scanf("%d", &avail[i]);

    for(i = 0; i < n; i++)
        finish[i] = 0;

    int count = 0;

    while(count < n)
    {
        int found = 0;

        for(i = 0; i < n; i++)
        {
            if(finish[i] == 0)
            {
                int possible = 1;

                for(j = 0; j < m; j++)
                {
                    if(req[i][j] > avail[j])
                    {
                        possible = 0;
                        break;
                    }
                }

                if(possible)

```

```

        {
            for(k = 0; k < m; k++)
                avail[k] += alloc[i][k];

            safe[count] = i;
            count++;
            finish[i] = 1;
            found = 1;
        }
    }
}

if(found == 0)
    break;
}

if(count == n)
{
    printf("\nSystem is in safe state.\n");
    printf("Safe Sequence is: ");

    for(i = 0; i < n; i++)
        printf("P%d ", safe[i]);
}
else
{
    printf("\nDeadlock detected.\n");
}

return 0;
}

```


Result:

```
C:\Users\nydhi\OneDrive\Doc  X + v
Name: NYDHILE NAGARAJ
USN:1WA24CS195

Enter number of processes: 3
Enter number of resources: 3
Enter allocation matrix:
Process 0: 2 2 2
Process 1: 2 1 3
Process 2: 3 2 4
Enter request matrix:
Process 0: 0 0 0
Process 1: 2 1 1
Process 2: 1 1 1
Enter available resources: 0 0 0

System is in safe state.
Safe Sequence is: P0 P1 P2
Process returned 0 (0x0)   execution time : 33.202 s
Press any key to continue.
```


b) Deadlock Algorithm

1. Let $work$ and $finish$ be vectors of length m and n
 - a) $work = available$
 - b) for $i \leftarrow 1$ to n , if $allocation_i \neq 0$, then $finish[i] = false$
2. find an index i such that
 - a) $finish[i] = false$
 - b) $request[i] \leq work$
 If no such i go to step 4
3. $work = work + allocation_i$
 $finish[i] = true$
 go to step 2
4. If $finish[i] == false$ for some i $1 \leq i \leq n$ then system is in deadlock state.
 If is in deadlock

Ex: Process Allocation Request work

P_0	102	001	000
P_1	211	102	
P_2	103	000	
P_3	122	330	

CODE

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, m, i, j, k;
```

```
    printf("Enter the number of processes: ");
```

```
    scanf("%d", &n);
```

b) Deadlock Algorithm

1. Let $work$ and $finish$ be vectors of length m and n
 - a) $work = available$
 - b) for $i \leftarrow 1$ to n , if $allocation_i \neq 0$, then $finish[i] = false$
2. find an index i such that
 - a) $finish[i] = false$
 - b) $request[i] \leq work$
 if no such i go to step 4
3. $work = work + allocation_i$
 $finish[i] = true$
 go to step 2
4. If $finish[i] == false$ for some i $1 \leq i \leq n$ then system is in deadlock state.
 It is in deadlock

Ex:

Process	Allocation	Request	work
P_0	102	001	000
P_1	211	102	
P_2	103	000	
P_3	122	320	

CODE

#include <stdio.h>

int main() {

int n, m, i, j, k;

printf("Enter the number of processes: ");

scanf("%d", &n);

```

printt ("Enter the number of resources:");
scanf ("%d", &n);
int alloc[n][m], request[n][m], avail[m];
int finish[n], safe[m];
printt ("Enter the allocation matrix:");
for (i=0; i<n; i++) {
    for (j=0; j<m; j++) {
        scanf ("%d", &alloc[i][j]); } }

```

```

printt ("Enter the request matrix: \n");
for (i=0; i<n; i++) {
    for (j=0; j<m; j++) {
        scanf ("%d", &request[i][j]); } }

```

```

print ("Enter the available resources: \n");
for (j=0; j<m; j++) {
    scanf ("%d", &avail[j]); }

```

```

for (i=0; i<n; i++) {
    finish[i] = 0; }
put count = 0;
while (count < n) {
    int found = 0;
    for (i=0; i<n; i++) {
        if (finish[i] == 0) {
            int possible = 1;
            for (j=0; j<m; j++) {
                if (request[i][j] > avail[j]) {
                    possible = 0;
                    break; } }

```



```

    for (k=0; k<=m; k++) {
        avail[k] += alloc[i][k];
    }
    safe[count++] = i;
    finish[i] = 1;
    found = 1; } } }
if (found == 0) {
    break; } }
if (count == n) {
    printf("system is in a safe state\n");
    printf("safe sequence is:");
    for (i=0; i<=n; i++) {
        printf("P %d", safe[i]);
        if (i != n-1)
            printf(" -> ");
    }
    else {
        printf("deadlock detected");
    }
    return 0; }

```

OUTPUT :

Enter the number of processes: 4
 Enter the number of resources: 3
 Enter the allocation matrix:

1 0 2

2 1 1

1 0 3

1 2 2

Enter request matrix :

0 0 1

1 0 2

0 0 0

3 3 30

Enter the available resources : 0 0 0

Program -6

Question:

Write a C program to simulate the following contiguous memory allocation techniques. (Any one)

- a) Worst-fit
- b) Best-fit
- c) First-fit

CODE:

```
#include <stdio.h>

void firstFit(int blocks[], int m, int processes[], int n) {
    int alloc[n], used[m];
    for (int i = 0; i < m; i++)
        used[i] = 0;
    for (int i = 0; i < n; i++)
        alloc[i] = -1;

    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            if (!used[j] && blocks[j] >= processes[i]) {
                alloc[i] = j;
                used[j] = 1;
                break;
            }

    printf("\n--- First Fit ---\n");
    printf("Process No.\tProcess Size\tBlock No.\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t\t%d\t\t", i + 1, processes[i]);
        if (alloc[i] != -1)
            printf("%d\n", alloc[i] + 1);
        else
```

```

        printf("Not Allocated\n");
    }
}

void bestFit(int blocks[], int m, int processes[], int n) {
    int alloc[n], used[m];
    for (int i = 0; i < m; i++)
        used[i] = 0;
    for (int i = 0; i < n; i++)
        alloc[i] = -1;

    for (int i = 0; i < n; i++) {
        int best = -1;
        for (int j = 0; j < m; j++)
            if (!used[j] && blocks[j] >= processes[i])
                if (best == -1 || blocks[j] < blocks[best])
                    best = j;
        if (best != -1) {
            alloc[i] = best;
            used[best] = 1;
        }
    }

    printf("\n--- Best Fit ---\n");
    printf("Process No.\tProcess Size\tBlock No.\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t\t%d\t\t", i + 1, processes[i]);
        if (alloc[i] != -1)
            printf("%d\n", alloc[i] + 1);
        else
            printf("Not Allocated\n");
    }
}

```

```

}

void worstFit(int blocks[], int m, int processes[], int n) {
    int alloc[n], used[m];
    for (int i = 0; i < m; i++)
        used[i] = 0;
    for (int i = 0; i < n; i++)
        alloc[i] = -1;

    for (int i = 0; i < n; i++) {
        int worst = -1;
        for (int j = 0; j < m; j++)
            if (!used[j] && blocks[j] >= processes[i])
                if (worst == -1 || blocks[j] > blocks[worst])
                    worst = j;
        if (worst != -1) {
            alloc[i] = worst;
            used[worst] = 1;
        }
    }

    printf("\n--- Worst Fit ---\n");
    printf("Process No.\tProcess Size\tBlock No.\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t%d\t", i + 1, processes[i]);
        if (alloc[i] != -1)
            printf("%d\n", alloc[i] + 1);
        else
            printf("Not Allocated\n");
    }
}

```



```

int main() {

    printf("Name: NYDHILE NAGARAJ\nUSN:1WA24CS195\n\n");

    int m, n;

    printf("Enter number of memory blocks: ");
    scanf("%d", &m);
    int blocks[m];
    printf("Enter sizes of %d memory blocks:\n", m);
    for (int i = 0; i < m; i++)
        scanf("%d", &blocks[i]);

    printf("Enter number of processes: ");
    scanf("%d", &n);
    int processes[n];
    printf("Enter sizes of %d processes:\n", n);
    for (int i = 0; i < n; i++)
        scanf("%d", &processes[i]);

    firstFit(blocks, m, processes, n);
    bestFit(blocks, m, processes, n);
    worstFit(blocks, m, processes, n);

    return 0;
}

```

Result:

```
C:\Users\nydhil\OneDrive\Doc  X + v
Enter sizes of 5 memory blocks:
100 200 300 400 500
Enter number of processes: 4
Enter sizes of 4 processes:
212 417 112 426

--- First Fit ---
Process No.      Process Size      Block No.
1                212                3
2                417                5
3                112                2
4                426              Not Allocated

--- Best Fit ---
Process No.      Process Size      Block No.
1                212                3
2                417                5
3                112                2
4                426              Not Allocated

--- Worst Fit ---
Process No.      Process Size      Block No.
1                212                5
2                417              Not Allocated
3                112                4
4                426              Not Allocated

Process returned 0 (0x0)   execution time : 28.854 s
Press any key to continue.
```

First fit

Process no	Process size	Process block
1	212	1
2	512	2
3	312	5
4	526	Not allocated

Best fit

Process No	Process size	Process Block
1	212	4
2	512	5
3	312	1
4	526	2

Worst fit

Process no	Process size	Process block
1	212	2
2	512	5
3	312	1
4	526	Not allocated

CODE :

```
#include <stdio.h>

void firstFit (int blocks[], int m, int
processes[], int n) {
    int allocation[n];
    for (int i = 0; i < n; i++)
        allocation[i] = -1;
    for (int i = 0; i < n; i++) {
```

classmate
Date _____
Page _____

```

for(int j=0; j<m; j++) {
    for(int i=0; i<n; i++) {
        if(blocks[j] >= process[i]) {
            allocation[i] = j;
            blocks[j] -= process[i];
            break; } }
printf("\n First fit allocation: \n");
printf("Process no. \t Process size \t Block no. \n");
for(int i=0; i<n; i++) {
    printf("%d \t %d \t %d", i+1, process[i],
    if(allocation[i] != -1)
        printf("%d", allocation[i]+1);
    else printf("Not allocated"); } }

```

```

void bestFit(int blocks[], int m, int process[],
int allocation[]) {
    for(int i=0; i<n; i++) { allocation[i] = -1;
    for(int i=0; i<n; i++) { int best = -1;
        for(int j=0; j<m; j++) {
            if(blocks[j] >= process[i]) {
                if(best == -1 || blocks[j] < blocks[best]) {
                    best = j; } } }
            if(best != -1) {
                allocation[i] = best;
                blocks[best] -= process[i]; } }
printf("\n Best fit allocation");
printf("Process no. \t Process size \t Block no. \n");
for(int i=0; i<n; i++) {
    printf("%d \t %d \t %d", i+1, process[i],
    if(allocation[i] != -1) {

```

```

printf ("%d\n", allocation[i] + 1);
else printf ("Not allocated"); }

void worstFit (int blocks[], int m, int
               processes[], int n) {
    int allocation[n];
    for (int i = 0; i < n; i++)
        allocation[i] = -1;
    for (int i = 0; i < n; i++) {
        int worst = -1;
        for (int j = 0; j < m; j++) {
            if (blocks[j] >= processes[i]) {
                if (worst == -1 || blocks[j] > blocks[worst]) {
                    worst = j; } } }
        if (worst != -1) {
            allocation[i] = worst;
            blocks[worst] -= processes[i];
            printf ("\n worst fit allocation: \n");
            printf ("process no \t process size \t Block No \n");
            for (int i = 0; i < n; i++) {
                printf (int i = 0; i < n; i++) {
                    if (allocation[i] != -1)
                        printf ("%d\n", allocation[i] + 1);
                    else
                        printf ("Not allocated \n"); } }

```


Date _____
Page _____

```

int main () {
    int memoryBlocks[] = {100, 500, 200, 300, 600};
    int processes[] = {212, 417, 112, 426};
    int m = sizeof(memoryBlocks) / sizeof(memoryBlocks[0]);
    int n = sizeof(processes) / sizeof(processes[0]);

    int blocks1[m], blocks2[m], blocks3[m];

    for (int i = 0; i < m; i++) {
        blocks1[i] = memoryBlocks[i];
        blocks2[i] = memoryBlocks[i];
        blocks3[i] = memoryBlocks[i];
    }

    firstFit(blocks1, m, processes, n);
    bestFit(blocks2, m, processes, n);
    worstFit(blocks3, m, processes, n);

    return 0;
}

```


Program -7

Question:

Write a C program to simulate page replacement algorithms. (Any one)

- a) FIFO
- b) LRU
- c) Optimal

FIFO

CODE:

```
#include <stdio.h>
```

```
int main() {
```

```
    printf("Name: NYDHILE NAGARAJ\nUSN: 1WA24CS195\n\n");
```

```
    int pages[20], frames[10];
```

```
    int n, f, i, j, k = 0, fault = 0, found;
```

```
    printf("Enter number of pages: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter page reference string: ");
```

```
    for(i = 0; i < n; i++) {
```

```
        scanf("%d", &pages[i]);
```

```
    }
```

```
    printf("Enter number of frames: ");
```

```
    scanf("%d", &f);
```

```
    for(i = 0; i < f; i++) {
```

```
        frames[i] = -1;
```

```
    }
```

```

for(i = 0; i < n; i++) {

    found = 0;

    for(j = 0; j < f; j++) {
        if(frames[j] == pages[i]) {
            found = 1;
            break;
        }
    }

    if(found == 0) {
        frames[k] = pages[i];
        k = (k + 1) % f;
        fault++;
    }

    printf("\nPage %d -> ", pages[i]);

    for(j = 0; j < f; j++) {
        if(frames[j] != -1)
            printf("%d ", frames[j]);
        else
            printf("- ");
    }

    printf("\n\nTotal Page Faults = %d\n", fault);

    return 0;
}

```

Result:

```
C:\Users\nydhi\OneDrive\Doc  X + v
Name: NYDHILE NAGARAJ
USN: 1WA24CS195

Enter number of pages: 12
Enter page reference string: 7 0 1 3 4 7 8 2 9 0 8 9 2 3
Enter number of frames:
Page 7 -> 7 -
Page 0 -> 7 0
Page 1 -> 1 0
Page 3 -> 1 3
Page 4 -> 4 3
Page 7 -> 4 7
Page 8 -> 8 7
Page 2 -> 8 2
Page 9 -> 9 2
Page 0 -> 9 0
Page 8 -> 8 0
Page 9 -> 8 9

Total Page Faults = 12

Process returned 0 (0x0)  execution time : 23.708 s
Press any key to continue.
```

#include <stdio.h>

```
int search (int page, int frames[], int f) {  
    int i;  
    for (i=0; i<n; i++) {  
        if (frames[i] == page);  
        return 1; }  
    return 0; }
```

```
void printFrames (int frames[], int f) {  
    int i;  
    printf ("[" );  
    for (i=0; i<f; i++) {  
        if (frames[i] == -1)  
            printf (" ");  
        else  
            printf ("%d", frames[i]);  
        if (i != f-1)  
            printf (" ");  
    }  
    printf ("]\n"); }
```

```
void FIFO (int pages[], int n, int f) {  
    int frames[f];  
    int i, pointer = 0, faults = 0;  
    for (i=0; i<f; i++)  
        frames[i] = -1;  
    printf ("\n* FIFO page replacement\n");  
    for (i=0; i<n; i++) {  
        printf ("page %d -> ", pages[i]);  
        if (! search (pages[i], frames, f)) {  
            frames[pointer] = pages[i];  
            pointer++;  
            faults++;  
        }  
    }  
    printf ("\nTotal faults: %d", faults);  
}
```

LRU

CODE:

```
#include <stdio.h>
```

```
int main() {
```

```
    printf("Name: NYDHILE NAGARAJ\nUSN: 1WA24CS195\n\n");
```

```
    int pages[20], frames[10], time[10];
```

```
    int n, f, i, j, pos, fault = 0, count = 0, found;
```

```
    printf("Enter number of pages: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter page reference string: ");
```

```
    for(i = 0; i < n; i++) {
```

```
        scanf("%d", &pages[i]);
```

```
    }
```

```
    printf("Enter number of frames: ");
```

```
    scanf("%d", &f);
```

```
    for(i = 0; i < f; i++) {
```

```
        frames[i] = -1;
```

```
    }
```

```
    for(i = 0; i < n; i++) {
```

```
        found = 0;
```

```
        for(j = 0; j < f; j++) {
```

```

    if(frames[j] == pages[i]) {
        count++;
        time[j] = count;
        found = 1;
        break;
    }
}

if(found == 0) {

    pos = 0;

    for(j = 1; j < f; j++) {
        if(time[j] < time[pos]) {
            pos = j;
        }
    }

    frames[pos] = pages[i];
    count++;
    time[pos] = count;
    fault++;
}

printf("\nPage %d -> ", pages[i]);

for(j = 0; j < f; j++) {
    if(frames[j] != -1)
        printf("%d ", frames[j]);
    else
        printf("- ");
}

```



```

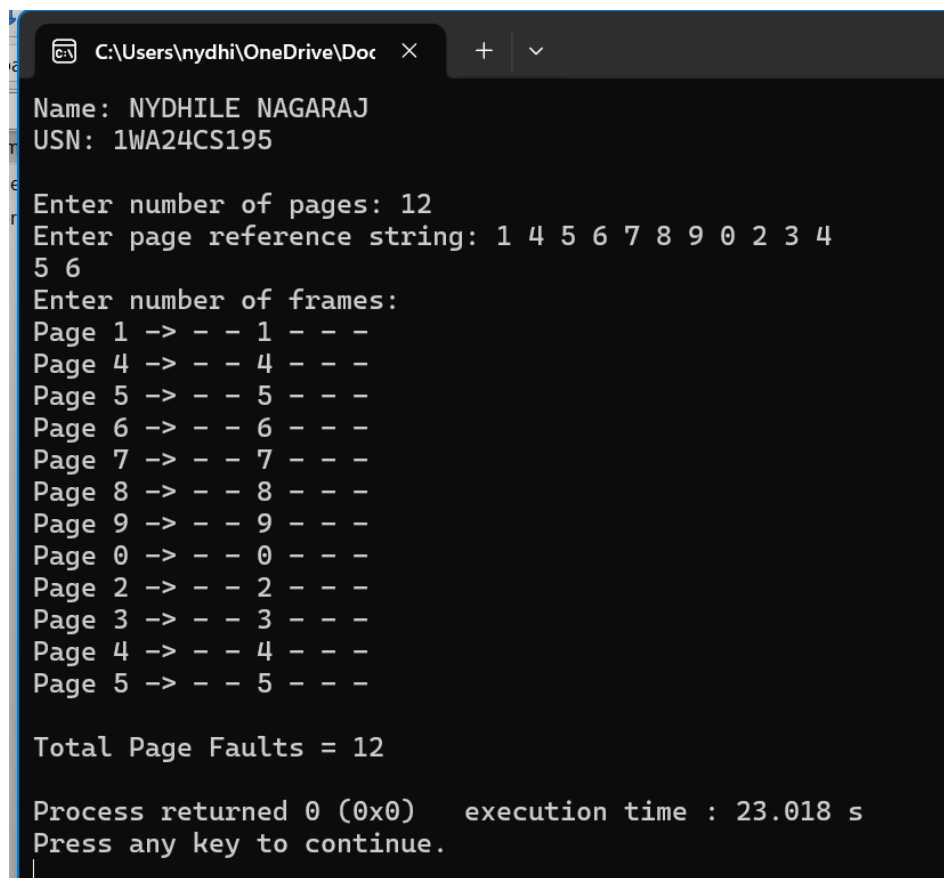
    }

    printf("\n\nTotal Page Faults = %d\n", fault);

    return 0;
}

```

Result:



```

C:\Users\nydhi\OneDrive\Doc >
Name: NYDHILE NAGARAJ
USN: 1WA24CS195

Enter number of pages: 12
Enter page reference string: 1 4 5 6 7 8 9 0 2 3 4
5 6
Enter number of frames:
Page 1 -> - - 1 - - -
Page 4 -> - - 4 - - -
Page 5 -> - - 5 - - -
Page 6 -> - - 6 - - -
Page 7 -> - - 7 - - -
Page 8 -> - - 8 - - -
Page 9 -> - - 9 - - -
Page 0 -> - - 0 - - -
Page 2 -> - - 2 - - -
Page 3 -> - - 3 - - -
Page 4 -> - - 4 - - -
Page 5 -> - - 5 - - -

Total Page Faults = 12

Process returned 0 (0x0)   execution time : 23.018 s
Press any key to continue.

```

```

    pointer = (pointer + 2) % P;
    faults++;
    printFrames(frames, f);
    printf("\n Total page faults (FIFO) : %d\n", faults);
}

void LRU(int pages[], int n, int f) {
    int frames[10], recent[10];
    int i, j, last, pos, faults = 0, count = 0;
    for (i = 0; i < n; i++) {
        frames = -1;
        recent[i] = 0;
        printf("LRU page replacement\n");
        for (i = 0; i < n; i++) {
            printf("Page %d -> ", pages[i]);
            if (search(pages[i], frames, f)) {
                for (j = 0; j < f; j++) {
                    if (frames[j] == pages[i]) {
                        recent[j] = ++count;
                        break;
                    }
                }
            } else {
                faults++;
                for (j = 0; j < f; j++) {
                    if (frames[j] == -1) {
                        frames[j] = pages[i];
                        recent[j] = ++count;
                        break;
                    }
                }
            }
            if (j == f) {
                last = recent[0];
                pos = 0;
                for (j = 1; j < f; j++) {

```

OPTIMAL PAGE REPLACEMENT

CODE:

```
#include <stdio.h>

int main() {

    printf("Name: NYDHILE NAGARAJ\nUSN: 1WA24CS195\n\n");

    int pages[20], frames[10];
    int n, f, i, j, k, pos, fault = 0, found;

    printf("Enter number of pages: ");
    scanf("%d", &n);

    printf("Enter page reference string: ");
    for(i = 0; i < n; i++) {
        scanf("%d", &pages[i]);
    }

    printf("Enter number of frames: ");
    scanf("%d", &f);

    for(i = 0; i < f; i++) {
        frames[i] = -1;
    }

    for(i = 0; i < n; i++) {

        found = 0;
```

```

for(j = 0; j < f; j++) {
    if(frames[j] == pages[i]) {
        found = 1;
        break;
    }
}

if(found == 0) {

    pos = -1;

    for(j = 0; j < f; j++) {

        int future = 0;

        for(k = i + 1; k < n; k++) {
            if(frames[j] == pages[k]) {
                future = 1;
                break;
            }
        }

        if(future == 0) {
            pos = j;
            break;
        }
    }

    if(pos == -1)
        pos = 0;

    frames[pos] = pages[i];

```

```

        fault++;
    }

    printf("\nPage %d -> ", pages[i]);

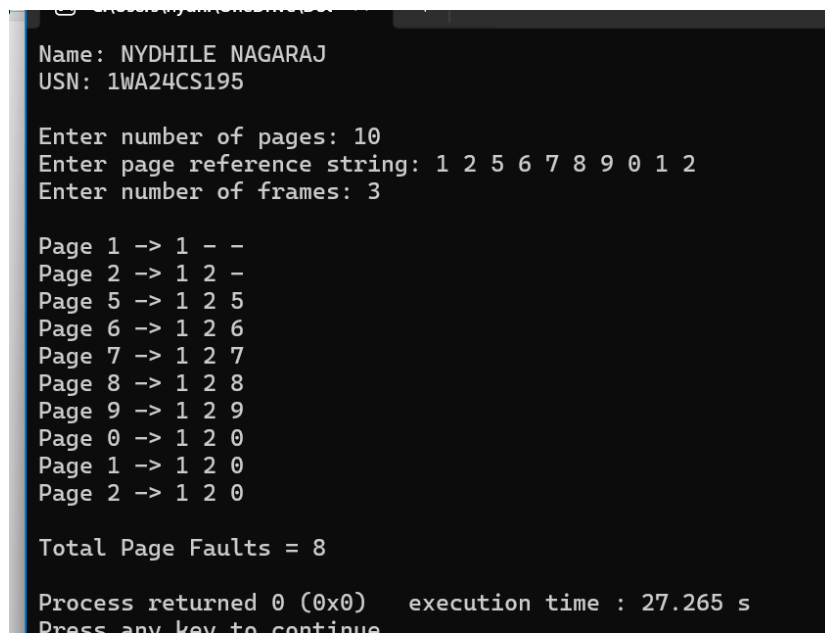
    for(j = 0; j < f; j++) {
        if(frames[j] != -1)
            printf("%d ", frames[j]);
        else
            printf("- ");
    }
}

printf("\n\nTotal Page Faults = %d\n", fault);

return 0;
}

```

Result:



```

Name: NYDHILE NAGARAJ
USN: 1WA24CS195

Enter number of pages: 10
Enter page reference string: 1 2 5 6 7 8 9 0 1 2
Enter number of frames: 3

Page 1 -> 1 - -
Page 2 -> 1 2 -
Page 5 -> 1 2 5
Page 6 -> 1 2 6
Page 7 -> 1 2 7
Page 8 -> 1 2 8
Page 9 -> 1 2 9
Page 0 -> 1 2 0
Page 1 -> 1 2 0
Page 2 -> 1 2 0

Total Page Faults = 8

Process returned 0 (0x0)   execution time : 27.265 s
Press any key to continue

```

```

if (recent[j] < least) { least = recent[j];
    pos = j; } }
frames[pos] = pages[i];
recent[pos] = ++count; } }
printFrames(frames, F); }
printf("\n total page faults (LRU): %d\n", faults);

void optimal(int pages[], int n, int f) {
    int frames[10];
    int i, j, k, pos, farthest, faults = 0;
    for (i = 0; i < n; i++) { frames[i] = -1; }
    printf("\n Optimal page replacement\n");
    for (i = 0; i < n; i++) {
        printf("page %d -> ", pages[i]);
        if (!search(pages[i], frames, f)) {
            faults++;
            for (j = 0; j < f; j++) {
                if (frames[j] == -1) {
                    frames[j] = pages[i];
                    break; } }
            if (j == f) { farthest = -1; pos = -1;
                for (j = 0; j < f; j++) { int found = 0;
                    for (k = i+1; k < n; k++) { if (frames[j]
                        found = 1; if (k > farthest) { farthest = k;
                            pos = j; } break; } }
                if (!found) { pos = j; break; } }
                frames[pos] = pages[i]; } }
            frames[pos] = pages[i]; } }
    printFrames(frames, f); }
    printf("\n Total page faults (optimal): ");

```



```

int n, f, i;
int pages[50];
printf("Enter no. of pages");
scanf("%d", &n);
printf("Enter page reference string: \n");
for (i=0; i<n; i++)
    scanf("%d", &pages[i]);
printf("Enter no. of frames:");
scanf("%d", &f);
FIFO(pages, n, f);
LRU(pages, n, f);
Optimal(pages, n, f);
return 0;
}

```

OUTPUT:

Enter no. of pages: 12

Enter page reference: 1 2 3 1 4

Enter no. of frames: 3

FIFO page replacement	LRU page replacement
page 1 → [1]	page 1 → [1]
page 2 → [1 2]	page 2 → [1 2]
page 3 → [1 2 3]	page 3 → [1 2 3]
page 1 → [1 2 3]	page 1 → [1 2 3]
page 4 → [4 2 3]	page 4 → [1 4 3]
total page faults: 4	total page faults (LRU): 4

16

optimal page replacement

page 1 → [1]	page 1 → [1 2 3]
page 2 → [1 2]	page 4 → [1 4 3]

NAME: Nydhile STD.: Sem3 SEC.: 30 ROLL NO.: 195 SUB.: Data Structure

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
1	27/2/26	Program 1: FCFS	1	10/2/26
2	27/2/26	Practice Program	3	
3	6/3/26	Program 1: FCFS	4-6	
4	6/3/26	Program 1: SJF ^{Non} preemptive	7-9	10/3/26
5	6/3/26	Program 1: SJF preemptive	9-11	
6	18/3/26	priority (non-preemptive)	12-14	
7	18/3/26	priority (preemptive)	15-17	10/3/26
8	18/3/26	Round robin	18-21	
9	17/3/26	multi level	22-24	
10	27/3/26	RMS		10/3/26
11	27/3/26	EDS		
12	27/3/26	proportional share		
13	24/4/26	5 Bounded Buffers	10	10/4/26
14	24/4/26	5 Dining Philosophers		
15	8/5/26	Safety algo	10	10/5/26
16	8/5/26	deadlock detection algo		
17	14/5/26	Best, Worst, Best case	10	10/5/26
18	22/5/26	LPU, FIFO, optimal	10	